

LUMI

Strategy & Feature Decisions Brief

The private, voice-first automation layer for professionals who can't use cloud AI

GDG Gemma Hackathon — Positioning, Innovation & Architecture Decisions
Compiled from working sessions, Aug 2026

1. Why This Document Exists

Lumi (rebuild of Trace v1) had strong functionality but no spine — chat, RAG, file-fetch, memory, tools, reminders, voice, device actions, and SOS sat next to each other as a feature list rather than one idea. Judged against general-purpose cloud LLMs on “knowledge and reasoning,” a 2B on-device model always loses, so that comparison was never winnable and the hackathon's own judging criteria don't reward it.

Hackathon judging weights (confirmed): Gemma Integration 30%, Innovation & Impact 30%, Functionality 20%, Presentation & Writeup 20%.

This document captures the repositioning work done across several sessions: where the app was under-differentiated, what research showed about what actually wins, and the concrete feature and architecture decisions made as a result.

2. The Repositioning

2.1 What Winning Gemma Projects Actually Look Like

Research into the Gemma 3n Impact Challenge and Gemma 4 Good Hackathon winners surfaced one consistent pattern: every winning project is a specific, structurally-excluded user, solving one task through a non-text modality, that a cloud LLM genuinely cannot do for that user — not “does more things.” Examples found: an AI assistant for the visually impaired using a chest-mounted camera and voice-only interaction; an offline tool translating images into spoken instructions for people with cognitive disabilities; a healthcare-worker tool combining vision (handwritten form extraction) and audio (Hindi voice transcription) for rural ASHA workers with limited digital literacy.

The hackathon's own framing makes this explicit: judges are told to look for solutions that work in places with poor connectivity or under strict privacy requirements, and to reward performance improvements on a specific task rather than generic prompting.

2.2 The New Category

Lumi is not a knowledge-first chatbot and not a general productivity app. Lumi is a **voice-first, privacy-first automation layer for working professionals who cannot legally or practically send data to cloud AI** — doctors (patient notes), lawyers (client details), business professionals (meeting recordings, client data) — with students as a secondary audience via RAG/flashcards/mood journal.

Chat is the interface, not the product. The product is what fires without the user opening the app or typing: a routine, a voice-triggered document, a meeting turned into notes and action items, a mail search cross-referenced with local notes.

2.3 Monetization Framing

- One-time payment, not subscription — framed as a direct consequence of the architecture (no per-token API cost to pass on), not just a pricing choice.
- Target market validated by real usage: consumer AI-wearable and meeting-copilot hardware (see §3) proves working professionals already pay for this category — Lumi delivers it without hardware cost or recurring fees.

3. Competitive Research: What Existing Products Get Wrong

Research covered two adjacent hardware-plus-subscription categories that validate the problem space and reveal a consistent structural weakness Lumi is positioned to exploit.

3.1 AI Recording Wearables (Limitless, Plaud, Bee, Omi, Friend)

Key findings — usable directly in the pitch

Consolidation risk: Limitless was acquired by Meta (Dec 2025) and stopped selling new hardware; Bee was acquired by Amazon. The two leading personal-recording wearables are now owned by the companies with the least credibility on personal data privacy.

Consent is unsolved: Limitless built a voice-ID “Consent Mode” to pause recording for new speakers, but reviewers note the ethical/legal burden of consent still falls entirely on the wearer, and these devices lack any universal indicator for people being recorded.

Subscription lock even after hardware purchase: several pendants add \$12–19/month tiers for full AI processing; one review frames this directly as “Subscription Anxiety to access your own memories.”

Known accuracy weakness: Bee Pioneer users report the AI frequently mislabels speakers or mixes up voices in multi-speaker meetings — worth testing Lumi’s own pipeline against this before the demo.

A reviewer’s own rhetorical question doubles as Lumi’s pitch line: “What can these devices do that your phone can’t? Your phone already has voice recording apps.”

3.2 On-Body Meeting/Conversation Copilots (Even Realities G2 “Conversate”, similar smart glasses)

A more directly relevant and newer category than the pendants: hardware that listens during a conversation, surfaces live prompts, and produces a post-meeting summary — already marketed to and used by professionals (a CEO uses it live on CNBC/Bloomberg interviews; a national deaf swimming team uses live translation to compete internationally).

Structural weaknesses found — Lumi’s direct wedge

Not actually private: even the camera-free, privacy-positioned G2 sends captured voice via Bluetooth to the Even Realities app, which uses the cloud to translate/process in real time. “Privacy-first” hardware is still shipping conversations to a server — unusable for genuinely confidential work despite the marketing.

Subscription-gated even after hardware purchase: on the prior-gen G1, the Translate feature required an additional Pro subscription on top of the \$599+ hardware cost.

Still phone-dependent: reviewers note that despite the hardware, many advanced features still require picking up and interacting with the phone — undermining the hands-free pitch.

“Prep notes” feature (upload documents ahead of a meeting; AI surfaces relevant context as concepts come up) is a genuinely good idea worth adopting — see §4.5.

Lumi’s position: the only version of this product category that is actually private, because inference never leaves the device — not “private” as a marketing claim layered over a cloud backend.

4. New Features Decided in This Round

4.1 Live Meeting Assistant (Silent Listen → Proactive Prompt)

Lumi joins/listens during a call in a silent mode — it knows it is in listen mode and does not interject. After the meeting ends, it proactively prompts the user: “Shall I add these todos? Want a summary? Notes prepared — you mentioned a Friday 14th presentation, want a calendar event?” This is the strongest single differentiator in the new feature set: it is action-taking with judgment, not passive transcription.

Architecture (key decision)

- Sherpa-ONNX performs 100% of audio → text via streaming ASR during the call. Gemma never touches raw audio, which makes Gemma's hard 30-second audio-clip cap a non-issue for this feature entirely — this is a deliberate design choice, not an oversight, and worth stating explicitly in the writeup as evidence of engineering care.
- Transcript chunks are queued as Sherpa-ONNX produces them during the call.
- After the meeting ends, the full reassembled transcript is sent to Gemma as text, in one pass, for summarization / todo-extraction / calendar-event detection — comfortably within Gemma's 128K context window for a typical meeting length.
- Accepted tradeoff: a 2–4 minute processing lag after the meeting ends is fine. This removes the need for real-time-everything and is the decision that makes the whole pipeline tractable on a phone.
- Battery/thermal load is materially lower than continuous LLM inference would be, since only a lightweight streaming ASR model runs during the call — still worth one real test run on target hardware before the hackathon.

4.2 Notification-Reading → Proactive Suggestions

Lumi reads incoming notifications via Android's NotificationListenerService and suggests todos/reminders from them. Technically distinctive because most apps don't request this permission; directly serves the zero-friction thesis. Requires an explicit runtime permission grant in system settings — worth naming in the demo as a real, nontrivial integration rather than a checkbox.

4.3 Voice-Triggered Document Fetch

Say “pull up my ID” and Lumi surfaces it instantly — no app-opening. Useful for entry/attendance and similar on-the-spot document needs. (Carried over from earlier positioning work, reaffirmed under the new professional-user framing.)

4.4 Voice-Set Reminders/Routines → Home Screen Widget

Natural-language voice commands set reminders and routines that update a home-screen widget directly, entirely on-device.

4.5 Meeting Prep From Own Notes + On-Demand Mail/Slack Fetch (New)

Directly adapted from Even Realities' “prep notes” feature (§3.2), combined with Lumi's existing RAG pillar and a new one-way mail/Slack fetch tool. Concrete example given during design: “I have a meeting on the marketing strategy of Lumi v3 — check the docs on Lumi and any new mails about it, and make a prerequisite checklist and summary before I attend.” This single request chains RAG over local docs + on-demand mail search + synthesis into one output — a strong, fast demo beat since it doesn't require a real multi-minute meeting to show.

Architecture (key decisions)

- Gmail/Slack access is via MCP, strictly one-way fetch, on-demand only, read-only — no send/write capability. Same trust model as notifications and web search: explicit per-account opt-in, nothing auto-syncs, user can revoke.
- Gemma's inference stays entirely local — only the connector I/O touches the network to fetch message contents; the message data is never sent to a third-party AI provider, only read locally by Lumi. Worth stating explicitly to judges as the precise boundary.
- Search/filter scoping (e.g. “any new mails about X”) needs to be a constrained query, not a full-inbox fetch — Gemma constructs the search query via its native function-calling, same pattern as the web-search tool design (§4.6).

- This closes the loop established by the meeting-todo pattern (§4.1): propose an action, always confirm with the user before anything is written or sent — one consistent behavior across meeting todos, web search, and mail/Slack fetch, not a one-off exception.

4.6 Web Search — Security Design (Reaffirmed)

Web search is the one feature that touches the open internet, so it needs a deliberately narrow design to not undermine the core privacy pitch:

- Explicit trigger only — never a silent fallback when Gemma is uncertain about something in private local context (notes, meetings, docs).
- Search queries are generated by Gemma via structured function-calling as a narrow, generic string, and treated as untrusted output — checked locally before dispatch to catch accidental leakage of names, note content, or transcript fragments into the query.
- Document/note/meeting content is never directly interpolated into a search query — enforced in code (the tool only accepts a plain query string), not just by prompting Gemma to behave.
- Results returned from the web are treated as untrusted input on the way back in too: raw fetched content should not be allowed to autonomously trigger further tool calls (e.g. a calendar write) in the same turn — a human-confirmation step sits between reading web content and acting on it, consistent with §4.5.

5. Gemma Integration — The 30% Story

5.1 Native Structured Function-Calling (Key Decision)

Gemma 3 scored 6.6% on the τ 2-bench agentic tool-use benchmark — not usable for serious agent work. Gemma 4 scores 86.4%, via dedicated architecture (6 special tokens creating a structured tool-use lifecycle), not prompt-engineered JSON extraction. Migrating Lumi's tool-calling to this native system — rather than a prompt-hacked JSON approach — is the concrete, demoable answer to “deeper Gemma integration” for judging, and directly explains why routing and tool use are fast and reliable.

5.2 Native Multimodal Architecture

Gemma 4's vision and audio are not bolted-on separate encoders — image, audio, and text are processed through a single decoder-only transformer. Lumi's vision-notes and voice features are one model, one weight set, not stitched-together components — worth stating this explicitly in the pitch.

5.3 Known Ecosystem Risk — Do Not Attempt This Week

Fine-tuning Gemma 4 multimodal for on-device deployment: unproven, avoid before the demo

The official Gemma 4 vision-tower export path for on-device deployment (`litert_torch/model_ext/gemma4/vision_exportable.py`) is currently a documented stub raising `NotImplementedError` — the wider `litert-torch/litert-community` ecosystem is stuck at the same wall as of this research.

Separately, a July 2026 report from a Qualcomm x Google LiteRT hackathon team found that even a cleanly fine-tuned, cleanly exported Gemma 4 E2B model failed at inference time with an undocumented Jinja chat-template error — a failure that only surfaces when actually running inference, not when loading the model. The worst possible time to discover this is mid-demo.

Decision: use Gemma 4 as shipped (including its built-in vision/audio, which is stable and is what Trace v1's working vision feature already relies on). Do not attempt custom fine-tuning + on-device export this week.

6. Open Questions / Not Yet Decided

- Voice implementation has not started. Open question: does Gemma itself handle transcription for general voice commands, or does Sherpa-ONNX handle all ASR (as decided for the meeting-assistant pipeline in §4.1) with Gemma used only for summarization/structuring/intent parsing? Leaning toward the latter for consistency, not yet finalized.
- Whether Gemma constructs mail/Slack search queries autonomously or the user provides more explicit search terms — affects how much scoping/leakage-check logic is needed before the hackathon.
- Real-device battery/thermal testing for a 45–60 minute continuous Sherpa-ONNX session has not yet been run.

7. Explicitly Out of Scope for Lumi (This Sprint)

- SOS (flashlight signal, emergency location sharing) — existed in Trace v1, not built in Lumi. Decided to leave it out entirely: it doesn't fit the working-professional privacy/automation positioning and would split the pitch's identity in the 3-minute demo. If ever built, mention only as a one-line footnote that the automation engine generalizes beyond productivity — never in the core demo.
- Browser-agent style automation (à la Comet, Atlas, Copilot Mode) — a large, real, subscription-heavy category, but a different form factor and competitive set than Lumi's mobile-native, wearable-adjacent positioning. Considered and deliberately not pursued, to avoid diluting the core story.
- Any send/write capability for email or Slack — read-only fetch only, for this sprint and for the hackathon demo.